

GitHub Copilot Coding Agent: Operational Reference

GitHub Copilot Coding Agent — Operational Reference

A no-fluff, implementation-focused breakdown. Each item lists **what / where / why**.

1. Core Operating Premises

- **Two agent surfaces:**
- **Coding agent** — autonomous, issue → PR, runs remotely (GitHub Actions).
- **Agent mode** — interactive, multi-step, in-editor.
- **Instruction-driven** — behavior shaped by Markdown config files in the repo.
- **Immediate effect** — instruction files are used as soon as saved/committed.
- **Precedence** — nearest AGENTS.md in the directory tree wins.

2. Configuration File Layout

Path	Scope	Purpose
.github/copilot-instructions.md	Repo-wide	Primary project context (stack, conventions, handling)
.github/instructions/**/*.instructions.md	Path-scoped	Targeted rules via applyTo frontmatter
\$HOME/.copilot/copilot-instructions.md	Personal	User-level instructions
AGENTS.md (+ nested)	Directory-scoped	Agent instructions; nearest file takes precedence
.github/agents/*.agent.md	Repo	Custom agent personas
.github/prompts/*.prompt.md	Repo	Reusable prompt templates
.github/workflow-policy.md	Repo	Defines AI workflow (e.g. Analyze→Reproduce→Debug→Fix→Validate)
.github/workflows/copilot-setup-steps.yml	Repo	Environment/setup for the coding agent (GitHub Actions)
.vscode/mcp.json	Workspace	MCP server definitions
\$COPILOT_CUSTOM_INSTRUCTIONS_DIRS	Env var	Extra instruction directories

3. Custom Instructions

- **Repo-wide:** .github/copilot-instructions.md
- **Path-specific:** .github/instructions/<name>.instructions.md
- Use YAML frontmatter applyTo: to scope to file/dir globs.
- **Content rules:**
 - Markdown; concise.
 - Include only what Copilot **cannot infer** (conventions, gotchas, stack).
- **Bootstrap:** run /init in chat to generate a starter config.

4. Custom Agents

- **Location:** .github/agents/
- **Naming:** <name>.agent.md (e.g. release-docs.agent.md, debugger.agent.md).
- **Definition (YAML frontmatter):** role, scope, capabilities, guardrails; can specify preferred model.
- **Invoke:** /agent slash command in Copilot CLI.

5. Prompt Files

- **Location:** .github/prompts/
- **Naming:** <name>.prompt.md
- **Purpose:** Reusable, parameterized task templates.
- **Can specify:** preferred model in frontmatter.

6. Environment Setup (copilot-setup-steps.yml)

- **Location:** .github/workflows/copilot-setup-steps.yml
- **Purpose:** Pre-install deps/tools so the coding agent's ephemeral environment is ready.
- **Premise:** Coding agent runs on GitHub Actions infrastructure — this provisions it.

7. MCP (Model Context Protocol)

- **Config:** .vscode/mcp.json or project settings.
- **Purpose:** Extend Copilot with external tools/context (e.g. cloud resources, live data).
- **Best practice:** Limit enabled tools → faster, more relevant responses.

8. Context Provision (in-chat)

Syntax	References
#<file>	A specific file
#<folder>	A folder
#<symbol>	A code symbol
• Plus MCP servers for info beyond the codebase.	

9. Workflow Policy

- **Location:** .github/workflow-policy.md
- **Purpose:** Codifies a repeatable AI workflow.
- **Example chain:** Analyze → Reproduce → Debug → Fix → Validate → Review.

10. Best Practices

- Keep instruction files concise; focus on non-inferable info.
- Scope with applyTo patterns or language/folder-specific files.
- Limit enabled tools/MCP servers.
- Enforce Workspace Trust in VS Code.
- Use /init to seed config; refine over time.
- Write specific prompts ("Fix the null deref in X when Y"), not "Fix this."
- Choose the right surface: coding agent (well-defined, async, PR-producing) vs agent mode (interactive).

11. Editor Settings (VS Code)

- Enable inline suggestions + chat.
- Disable experimental pop-ups for stability.
- Enforce Workspace Trust before agent execution.